

Writable Root PATH

Introduction: Why Writable \$PATH Exploitation is Powerful

Writable directories in the \$PATH environment variable present a significant security risk because they allow attackers to inject malicious scripts or binaries that can be executed by privileged users. By modifying the \$PATH, an attacker can control which executables are prioritized when a command is executed. If the \$PATH is manipulated to point to a directory where the attacker has placed a malicious script, that script can be executed in place of the intended system command. This can lead to privilege escalation, data theft, and even remote code execution.

Exploiting writable directories within \$PATH is an effective attack method because it doesn't necessarily require direct access to modify system files. It leverages existing environment configurations to execute malicious code.

1. What folders are located under \$PATH?

Command	Description
echo \$PATH	Displays the current directories listed in the \$PATH environment variable.

By examining \$PATH, you can identify which directories are searched for executables when a command is run. If any of these directories are writable by the user, they become potential targets for placing malicious executables.

2. Does your current user have write privileges for any of these folders?

Command	Description
<code>find / -writable 2>/dev/null cut -d "/" -f 2,3 grep -v proc sort -u</code>	Searches for writable directories under \$PATH. It checks all directories and filters out /proc.

This step helps identify which directories within \$PATH are writable by the current user. These directories are critical to the exploitation process, as they allow you to insert scripts or binaries that will be executed when a command is run.

3. Can you modify \$PATH?

Command	Description
<code>export PATH=/tmp:\$PATH</code>	Temporarily modifies the \$PATH variable to prioritize /tmp directory, allowing potential malicious binaries to be executed first.

By modifying \$PATH, an attacker can prepend directories (such as /tmp) that they control, ensuring that any executables in those directories take precedence over system binaries.

4. Is there a script/application you can start that will be affected by this vulnerability?

Command	Description
<code>strings /usr/bin/uustat</code>	Scans the binary /usr/bin/uustat for potential vulnerabilities or clues about exploitable functions affected by a modified \$PATH.

Identifying vulnerable applications or scripts that will be executed under the altered \$PATH is key. Once you know which executables are vulnerable, you can place a malicious version of those binaries in a directory that you control.

Additional Steps to Consider

Step	Command/Description
Monitor binaries	Check if any executables under your \$PATH are vulnerable to manipulation.
Create a malicious script	Example: Place a reverse shell payload in /tmp or another writable directory.
Wait for execution	If a vulnerable application runs, it may invoke your malicious script due to the modified \$PATH.

Real-World Impact

Writable \$PATH directories can be a powerful vector for privilege escalation, especially in scenarios where users or applications execute commands with elevated privileges. For example, if an attacker can manipulate the \$PATH of a cron job, or in the context of a privileged user running a vulnerable binary, they can control the flow of execution. A seemingly benign command could trigger an attacker's payload instead of the legitimate application.

This technique is especially dangerous in environments where cron jobs or automated scripts are frequently executed with high-level permissions (like root). An attacker who gains access to a system with even low privileges can manipulate the \$PATH variable to gain full control over the system by executing arbitrary commands.
